

B.Tech. CSE Semester IV
BCO 081B PROGRAMMING WITH PYTHON

UNIT 5

Python Graphics & GUI

Prepared by: Tushar Vyas (Assistant Professor, CSE)

Python GUI

Python provides various options for developing graphical user interfaces (GUIs). Most important are listed below:

- **Tkinter** – Tkinter is the Python interface to the Tk GUI toolkit shipped with Python.
- **wxPython** – This is an open-source Python interface for wxWidgets library.
- **JPython** – JPython is a Python port for Java which gives Python scripts seamless access to Java class libraries on the local machine.

Tkinter Programming

- Tkinter is the standard GUI library for Python.
- Python when combined with Tkinter provides a fast and easy way to create GUI applications.
- Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit.
- Creating a GUI application using Tkinter involves the following steps:
 - Import the module – tkinter
 - Create the main window (container)
 - Add any number of widgets to the main window
 - Apply the event Trigger on the widgets.

Tkinter Programming

- Importing tkinter is same as importing any other module in the Python code:

import tkinter

- There are two main methods used which the user needs to remember while creating the Python application with GUI:

- **Tk():** To create a main window, tkinter offers a method 'Tk()'. The basic code used to create the main window of the application is:

m=tkinter.Tk() where m is the name of the main window object

- **mainloop():** This method is used when our application is ready to run. It is an infinite loop used to run the application, wait for an event to occur and process the event as long as the window is not closed

m.mainloop()

Tkinter Programming

- tkinter also offers access to the geometric configuration of the widgets which can organize the widgets in the parent windows.
- There are mainly three geometry manager classes:
 - **pack() method:**
It organizes the widgets in blocks before placing in the parent widget.
 - **grid() method:**
It organizes the widgets in grid (table-like structure) before placing in the parent widget.
 - **place() method:**
It organizes the widgets by placing them on specific positions directed by the programmer.

Tkinter Widgets

There are currently 19 types of widgets in Tkinter:

1. **Button:** The Button widget is used to display buttons in your application.
2. **Canvas:** The Canvas widget is used to draw shapes, such as lines, ovals, polygons and rectangles, in your application.
3. **Checkbutton:** The Checkbutton widget is used to display a number of options as checkboxes. The user can select multiple options at a time.
4. **Entry:** The Entry widget is used to display a single-line text field for accepting values from a user.
5. **Frame:** The Frame widget is used as a container widget to organize other widgets.
6. **Label:** The Label widget is used to provide a single-line caption for other widgets. It can also contain images.

Tkinter Widgets

7. **Listbox:** The Listbox widget is used to provide a list of options to a user.
8. **Menubutton:** The Menubutton widget is used to display menus in your application.
9. **Menu:** The Menu widget is used to provide various commands to a user. These commands are contained inside Menubutton.
10. **Message:** The Message widget is used to display multiline text fields for accepting values from a user.
11. **Radiobutton:** The Radiobutton widget is used to display a number of options as radio buttons. The user can select only one option at a time.
12. **Scale:** The Scale widget is used to provide a slider widget.

Tkinter Widgets

- 13. Scrollbar:** The Scrollbar widget is used to add scrolling capability to various widgets, such as list boxes.
- 14. Text:** The Text widget is used to display text in multiple lines.
- 15. Toplevel:** The Toplevel widget is used to provide a separate window container.
- 16. Spinbox:** The Spinbox widget is a variant of the standard Tkinter Entry widget, which can be used to select from a fixed number of values.
- 17. PanedWindow:** A PanedWindow is a container widget that may contain any number of panes, arranged horizontally or vertically.

Tkinter Widgets

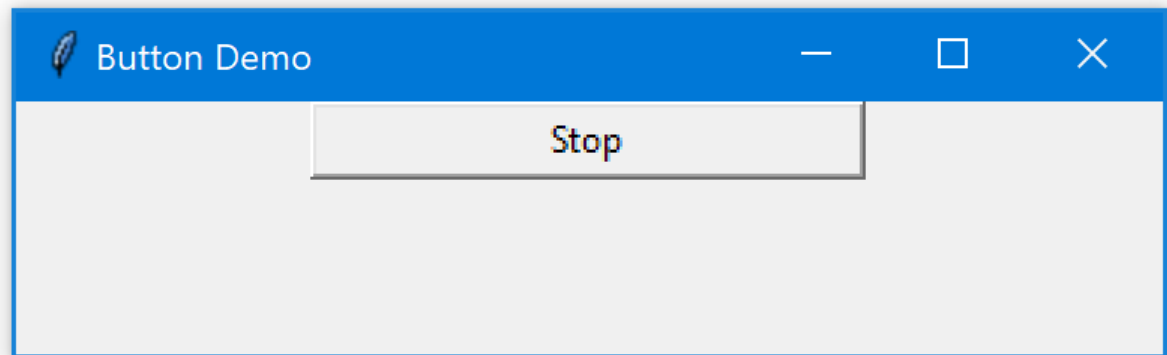
- 18. LabelFrame:** A labelframe is a simple container widget. Its primary purpose is to act as a spacer or container for complex window layouts.
- 19. tkMessageBox:** This module is used to display message boxes in your applications.

Button

Example

```
import tkinter as tk

r = tk.Tk()
r.title('Button Demo')
button = tk.Button(r, text='Stop', width=25, command=r.destroy)
button.pack()
r.mainloop()
```



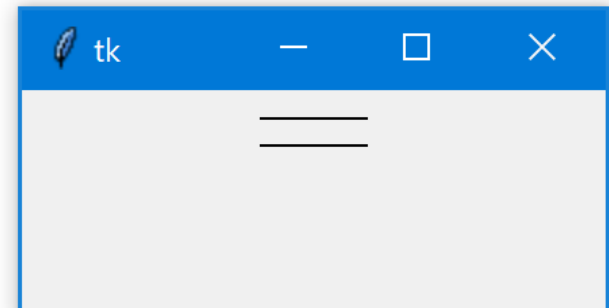
Canvas

Example

```
from tkinter import *  
r = Tk()  
w = Canvas(r, width=40, height=60)  
w.pack()  
canvas_height=20  
canvas_width=200  
y = int(canvas_height / 2)  
w.create_line(0, y, canvas_width, y )  //(x1,y1)to(x2,y2)  
w.create_line(0, y+10, canvas_width, y+10 )  
mainloop()
```

Other Shapes:

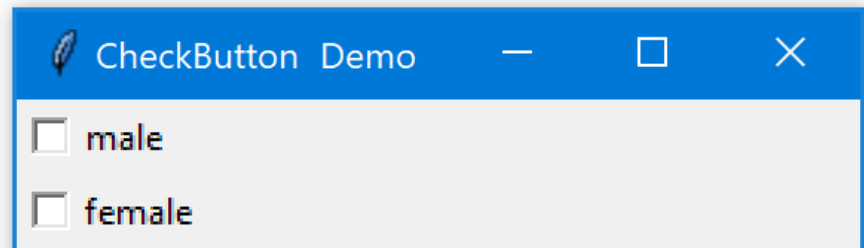
create_oval, create_rectangle,
create_arc, create_polygon



CheckButton

Example

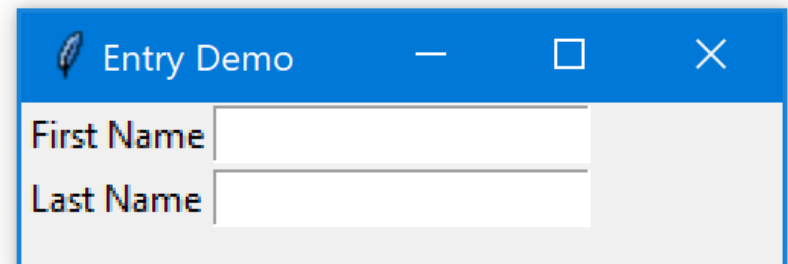
```
from tkinter import *  
r = Tk()  
r.title('CheckButton Demo')  
var1 = IntVar()  
Checkbutton(r, text='male', variable=var1).grid(row=0, sticky=W)  
var2 = IntVar()  
Checkbutton(r, text='female', variable=var2).grid(row=1, sticky=W)  
mainloop()
```



Entry

Example

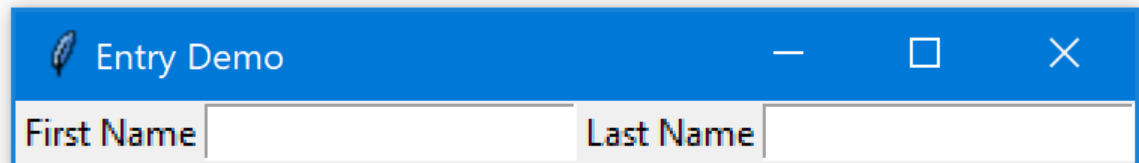
```
from tkinter import *  
r = Tk()  
r.title('Entry Demo')  
  
Label(r, text='First Name').grid(row=0)  
e1 = Entry(r)  
e1.grid(row=0, column=1)  
  
Label(r, text='Last Name').grid(row=1)  
e2 = Entry(r)  
e2.grid(row=1, column=1)  
mainloop()
```



Entry

Example

```
from tkinter import *  
r = Tk()  
r.title('Entry Demo')  
  
Label(r, text='First Name').grid(row=0, column=1)  
e1 = Entry(r)  
e1.grid(row=0, column=2)  
  
Label(r, text='Last Name').grid(row=0, column=3)  
e2 = Entry(r)  
e2.grid(row=0, column=4)  
mainloop()
```



Turtle Graphics in Python

- “Turtle” is a Python feature like a drawing board, which lets us command a turtle to draw all over it!
- The turtle module uses **tkinter** for the underlying graphics, it needs a version of Python installed with Tk support.
- Commonly used turtle methods are:

METHOD	PARAMETER	DESCRIPTION
Turtle()	None	It creates and returns a new turtle object
forward()	amount	It moves the turtle forward by the specified amount
backward()	amount	It moves the turtle backward by the specified amount
right()	angle	It turns the turtle clockwise
left()	angle	It turns the turtle counter clockwise

METHOD	PARAMETER	DESCRIPTION
penup()	None	It picks up the turtle's Pen
pendown()	None	Puts down the turtle's Pen
up()	None	Picks up the turtle's Pen
down()	None	Puts down the turtle's Pen
color()	Color name	Changes the color of the turtle's pen
fillcolor()	Color name	Changes the color of the turtle will use to fill a polygon
goto()	x, y	It moves the turtle to position x,y
dot()	None	Leaves the dot at the current position
stamp()	None	Leaves an impression of a turtle shape at the current location
shape()	shapename	Should be 'arrow', 'classic', 'turtle' or 'circle'

Plotting using Turtle

The roadmap for executing a turtle program follows 4 steps:

- Import the turtle module
- Create a turtle to control.
- Draw around using the turtle methods.
- Run `turtle.done()`

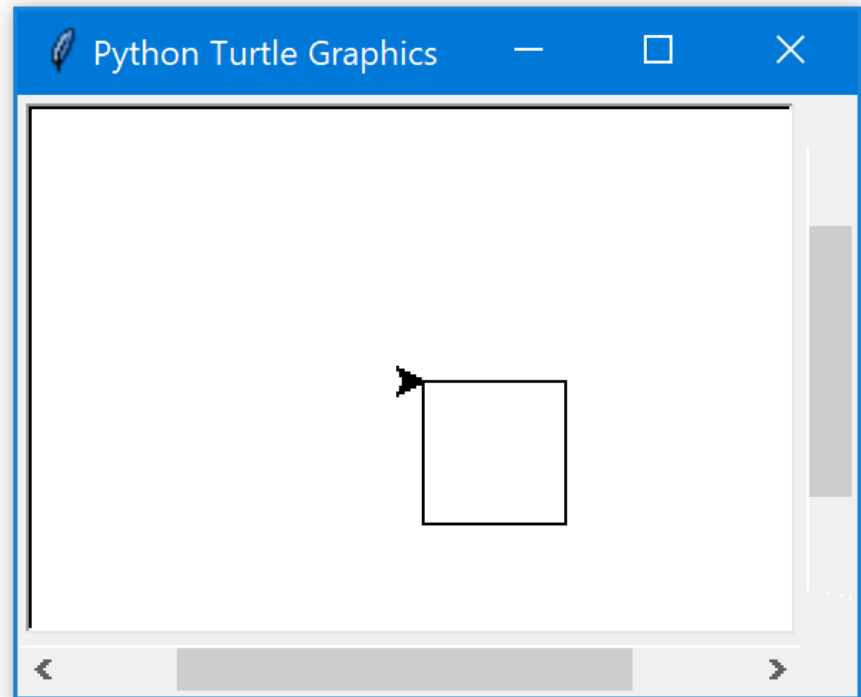
Shape 1: Square

Example

```
import turtle
skk = turtle.Turtle()

for i in range(4):
    skk.forward(50)
    skk.right(90)

turtle.done()
```



Shape 2: Star

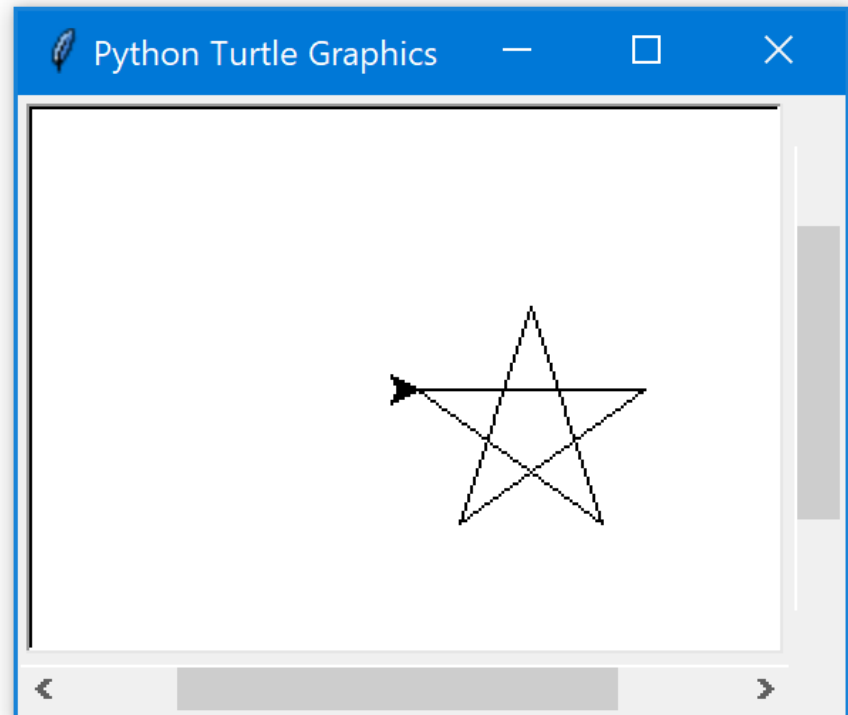
Example

```
import turtle

star = turtle.Turtle()

for i in range(5):
    star.forward(50)
    star.right(144)

turtle.done()
```



Shape 3: Hexagon

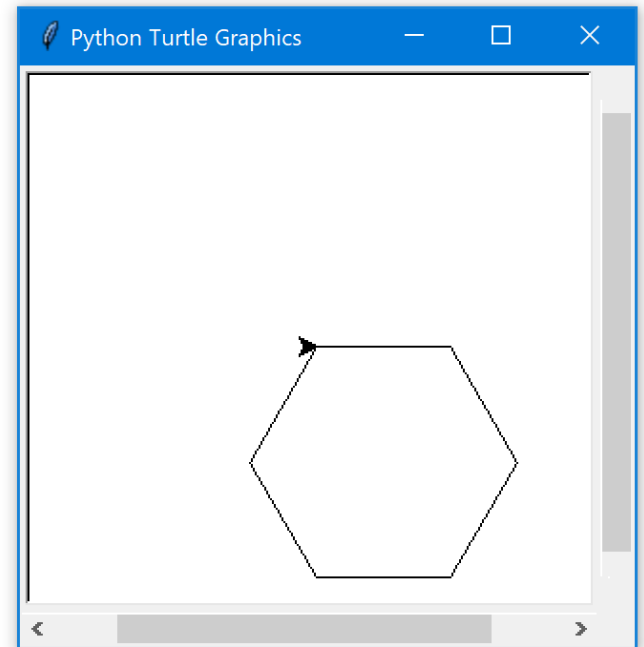
Example

```
import turtle
polygon = turtle.Turtle()

num_sides = 6
side_length = 70
angle = 360.0 / num_sides

for i in range(num_sides):
    polygon.forward(side_length)
    polygon.right(angle)

turtle.done()
```

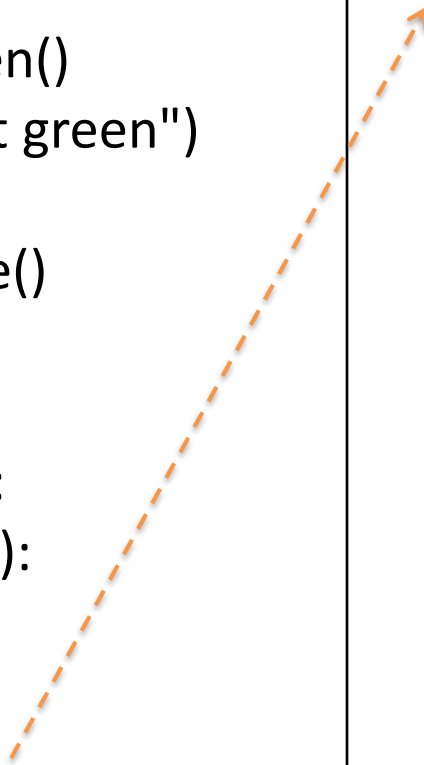


Shape 4: Square Spiral

Example

```
import turtle
wn = turtle.Screen()
wn.bgcolor("light green")
wn.title("Turtle")
skk = turtle.Turtle()
skk.color("blue")
```

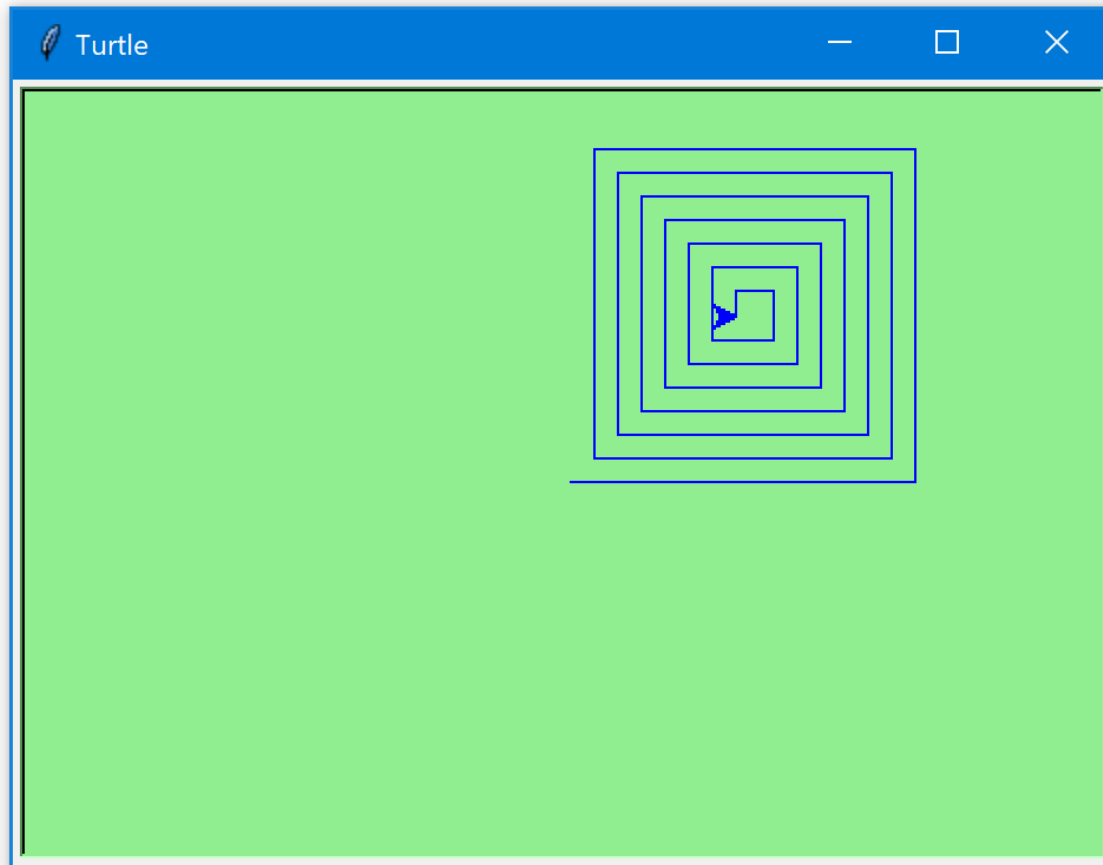
```
def sqrfunc(size):
    for i in range(4):
        skk.fd(size)
        skk.left(90)
        size = size-5
```



```
sqrfunc(146)
sqrfunc(126)
sqrfunc(106)
sqrfunc(86)
sqrfunc(66)
sqrfunc(46)
sqrfunc(26)
```

Shape 4: Square Spiral

Output



Shape 5: Olympic Rings

Example

```
import turtle
myTurtle = turtle.Turtle(shape="turtle")
myTurtle.width(10)

myTurtle.color('blue')
myTurtle.circle(50)
myTurtle.penup()
myTurtle.setposition(-120, 0)
myTurtle.pendown()
myTurtle.color('green')
myTurtle.circle(50)
myTurtle.penup()
myTurtle.setposition(60,60)
myTurtle.pendown()
```

Shape 5: Olympic Rings

Example

```
myTurtle.color('yellow')
myTurtle.circle(50)
myTurtle.penup()
myTurtle.setposition(-60, 60)
myTurtle.pendown()
myTurtle.color('red')
myTurtle.circle(50)
myTurtle.penup()
myTurtle.setposition(-180, 60)
myTurtle.pendown()
myTurtle.color('black')
myTurtle.circle(50)

turtle.done()
```

Shape 5: Olympic Rings

Output

